

Name: Ravitejas

SRN: PES1UG24CS603

Problem Statement 3: Campus Placement & Internship Pipeline

Lab 3: Component Modelling & Architectural Pattern Selection

Architectural Decision Record

1. Architecture Selection

We chose a **Layered Architecture (with Service-Oriented Business Components)** for the Campus Placement & Internship Pipeline System.

2. Architectural Choice Justification

When evaluating how to build this pipeline, my primary focus was on predictability and maintainability. Campus placements are high-stakes and time-sensitive for students, meaning the system cannot afford to be fragile.

I opted for a Layered Architecture for two practical reasons:

- **Strict Separation of UI and Heavy Compute:** In a real-world scenario, dashboard layouts and company eligibility rules change constantly. By strictly decoupling our Presentation Layer (the UIs) from the Business Logic Layer, we can push frontend updates without risking breaks to our core parsing engine. The UI simply consumes the `IDriveApplication` interface and remains completely blind to *how* the backend processes the heavy lifting.
- **Natural Fit for a Sequential Pipeline:** A placement drive is inherently a pipeline workflow: a student applies $\rightarrow$ their resume is parsed $\rightarrow$ they queue for interviews $\rightarrow$ an offer is issued. A Layered Architecture mirrors this top-down data flow perfectly. It allowed me to design distinct, modular services (like the `Placement Drive Manager` and `Interview Queue Manager`) that logically hand off data down to the infrastructure layer, making system failures much easier to isolate and debug.

3. Security Advantage: Attack Surface Reduction

When handling sensitive student academic records (PII), security is the top priority. The biggest security win with this layered approach is the **complete isolation of the Data Layer**.

The frontend components have zero direct access to the database. Instead, every data request must be routed through the Business Logic layer and forced through a single chokepoint: the `IDataPersistence` interface.

Because all database reads and writes happen exclusively through this interface, we can universally enforce our AES-256 encryption requirement (**NFR-001**) at a single tier. This centralized architecture virtually eliminates the risk of accidental plaintext data leaks from stray frontend queries.

4. Performance Benefit: Managing Peak Drive Traffic

Placement systems are notorious for experiencing massive traffic spikes right before a registration deadline. Our system must smoothly handle 500 simultaneous API requests per minute (**NFR-002**).

By compartmentalizing the backend into distinct service components, we successfully isolate our performance bottlenecks. For instance, the `Resume Parsing Engine` is computationally heavy (allowed up to 3.0 seconds per **FR-001**). Because it operates as a distinct component behind the `IResumeAnalysis` interface, we can independently scale it or assign it dedicated server threads during a traffic rush.

This isolation ensures that heavy PDF processing doesn't block the main application threads, keeping the Student Web Dashboard fast, responsive, and crash-free for everyone else.